

Python Training Day 1

Timings

Agenda

Fundamental Terms and Concepts

Installing and setting up python

Literals, Variables, and Numeral Systems

Input and Output in Python (Console I/O)

Operators and Expressions

9.30 to 11.00AM – p1

11 – 11.15 – break

11.15 to 1.30 – p2

1.30 to 2.30 – lunch break

2.30 to 3.45 – p3

3.45 to 4.00 – break

4.00 to 5.30 – p4

Have you ever wanted a solution

where you get data from various departments
you have to compile those, generate insights, manually create a report out
of it

to automate this?

Programming language

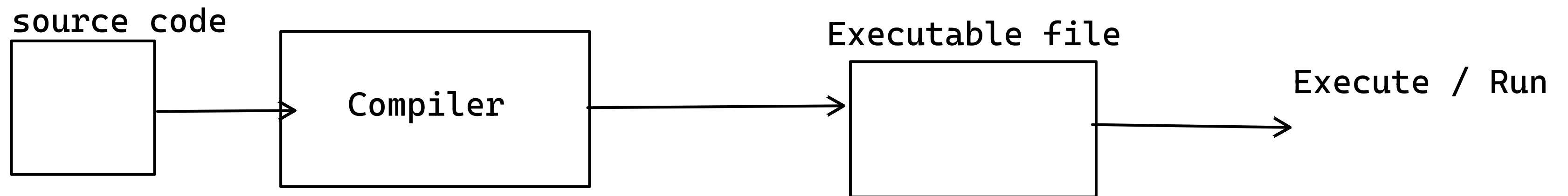
your High level English kind of language to machine instructions

There are 2 kinds of languages

compiled language

interpreted language

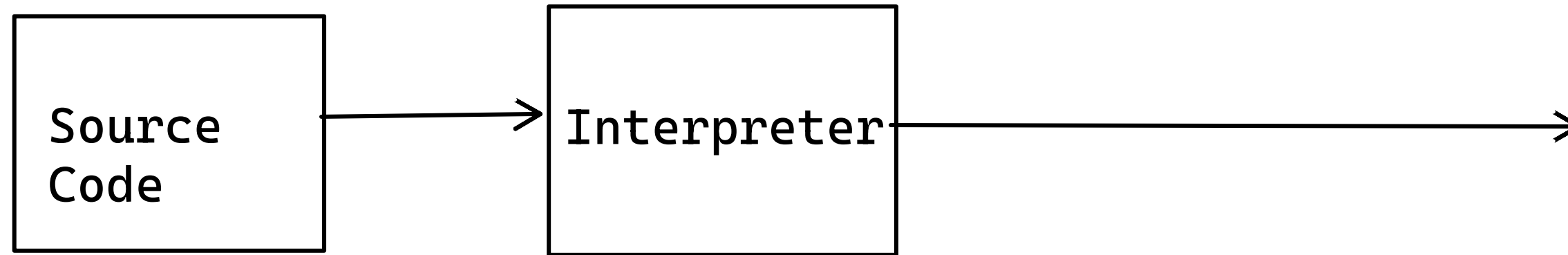
Compiled Language



Examples:

c, C++, Csharp

Interpreted Language



Script

Read each command (statement)
Convert that to 1/0
execute the command/ run the command

Examples:

Python

Javascript

Python Programming Language

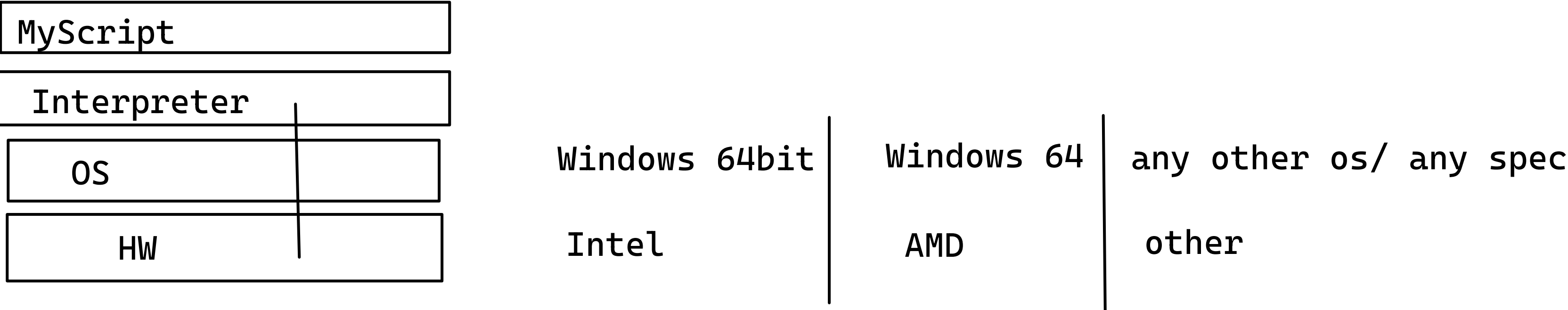
Guido Van Rossum

1. General purpose Language
2. Open Source language
3. Architecture Independent Language
4. Dynamically Typed Language (later)

1. I can perform – Web programming, system applications, Numerical Computation
2. Open Source Language:
 - Source Code is available, for free to use, modify & redistribute
 - Example: Linux (ubuntu, red hat, fedora), Android phone

Architecture Independence

MyScript can be run on any architecture without modifications



Java / Dotnet vs Python

- Its almost similar
- Python actually takes features from c, c++, Java
- they are Object oriented Language, but python is Functional programming Language

Interpreter of Python

- Needed to run a Python Script

Versions

- C Cython C Code + Python can be used
- Java Jython Java Code + Python
- R RPython R Code + Python
- Python Pypi

Versions of Python

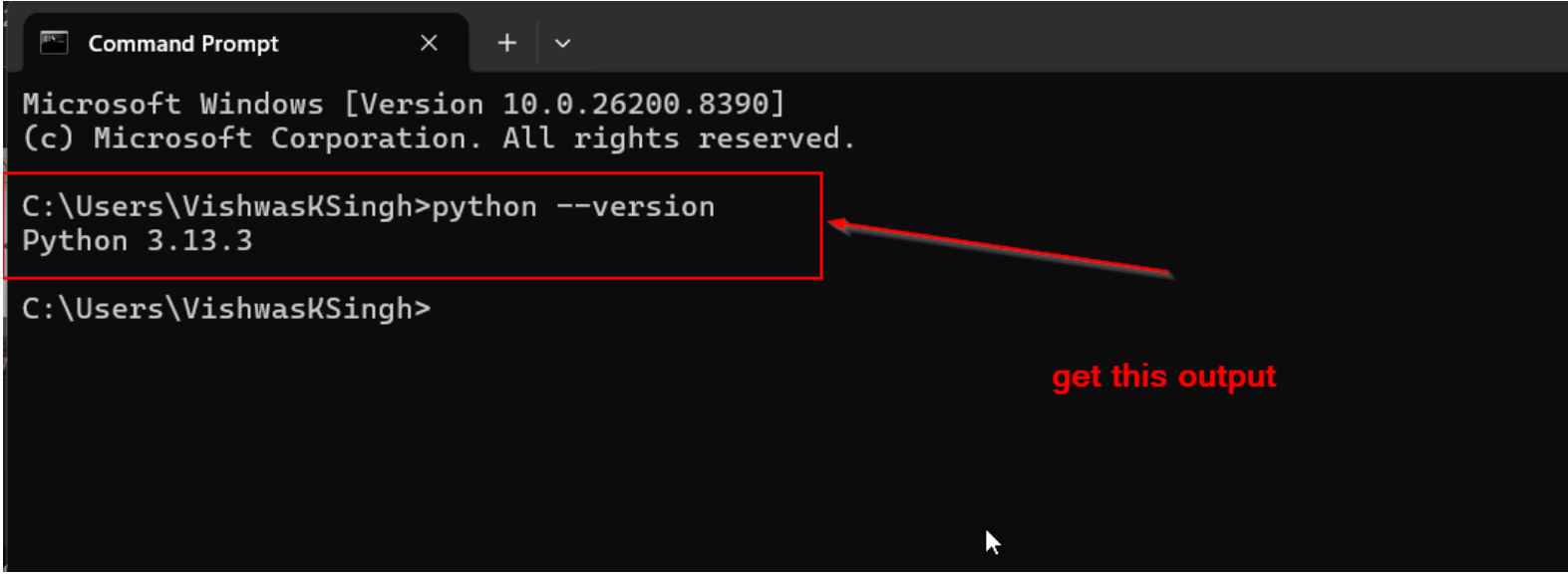
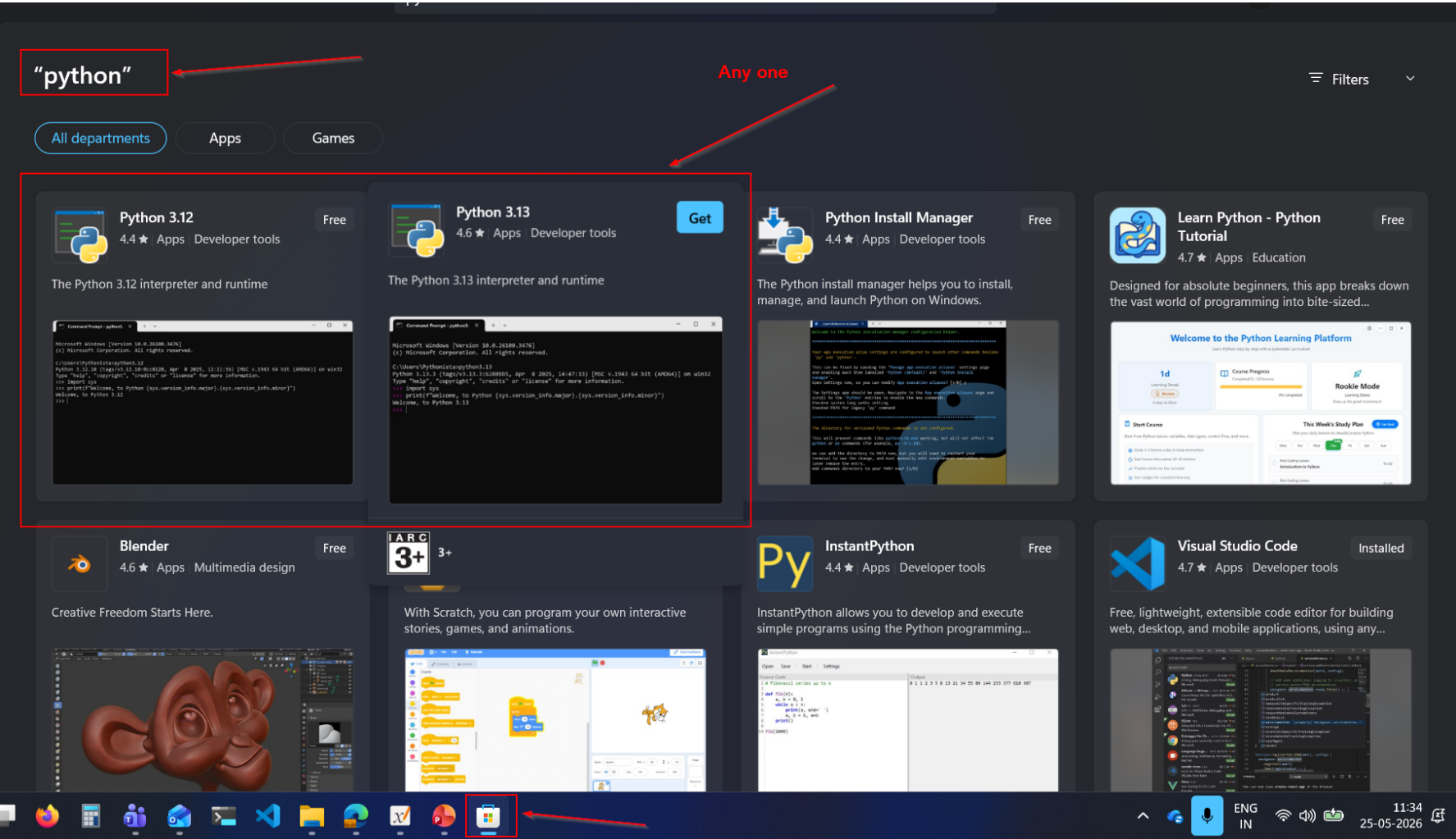
2.x.x

3.x.x (latest recommended)

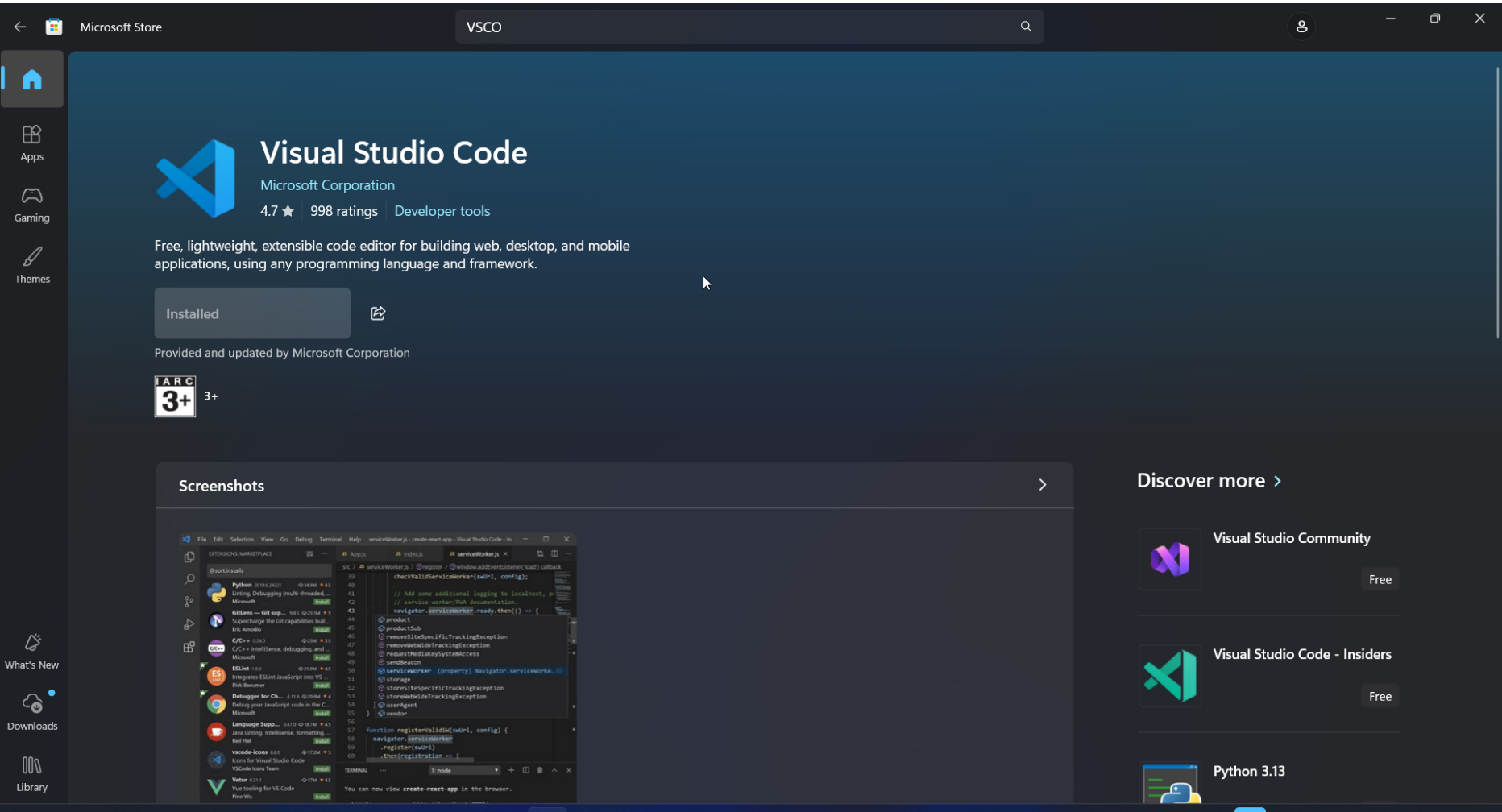
2.x  3.x python.org

3.9 — 3.12 — 3.15

Lab: Installation of Python

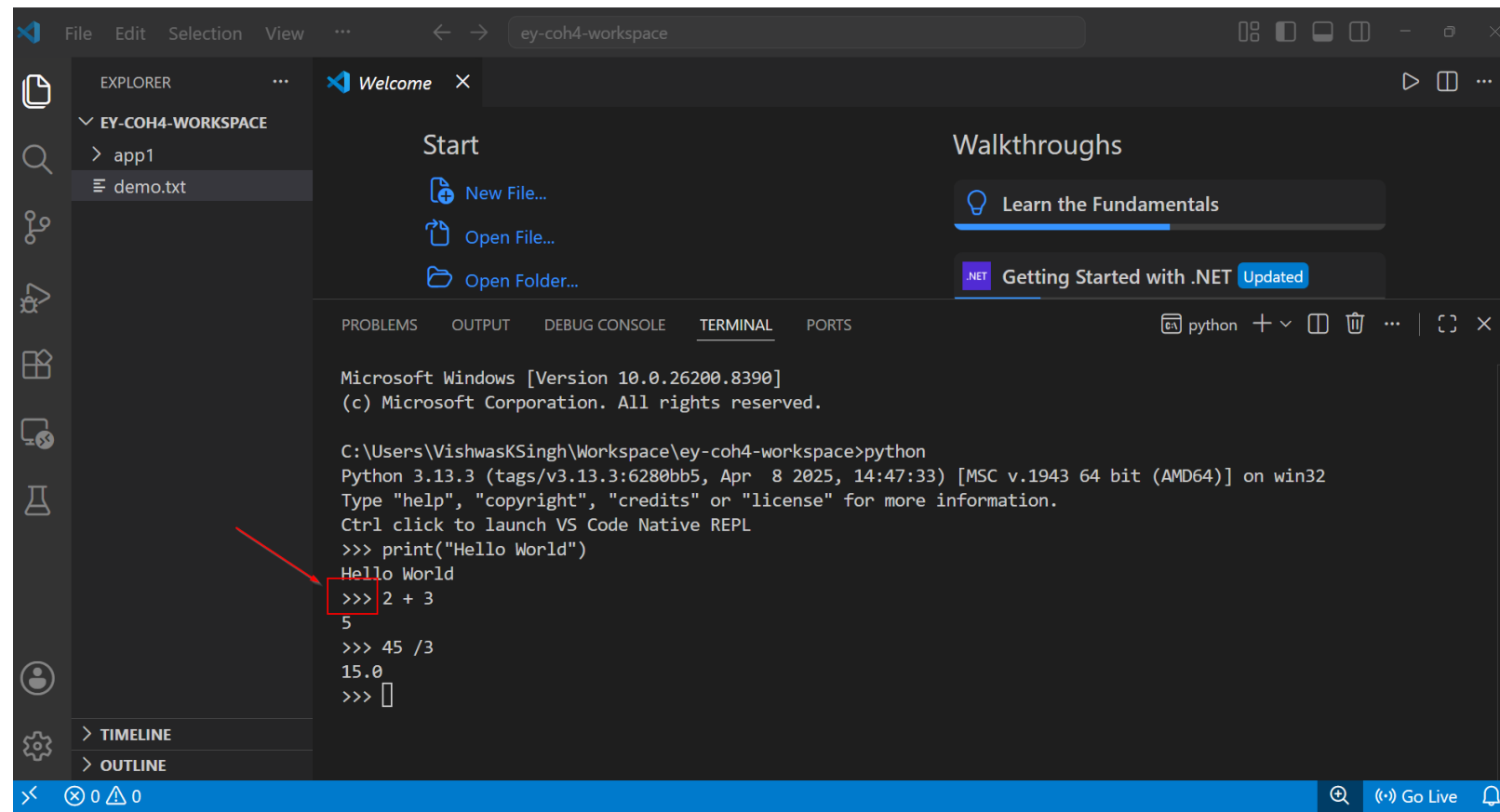


Lab: Text Editor: VS Code



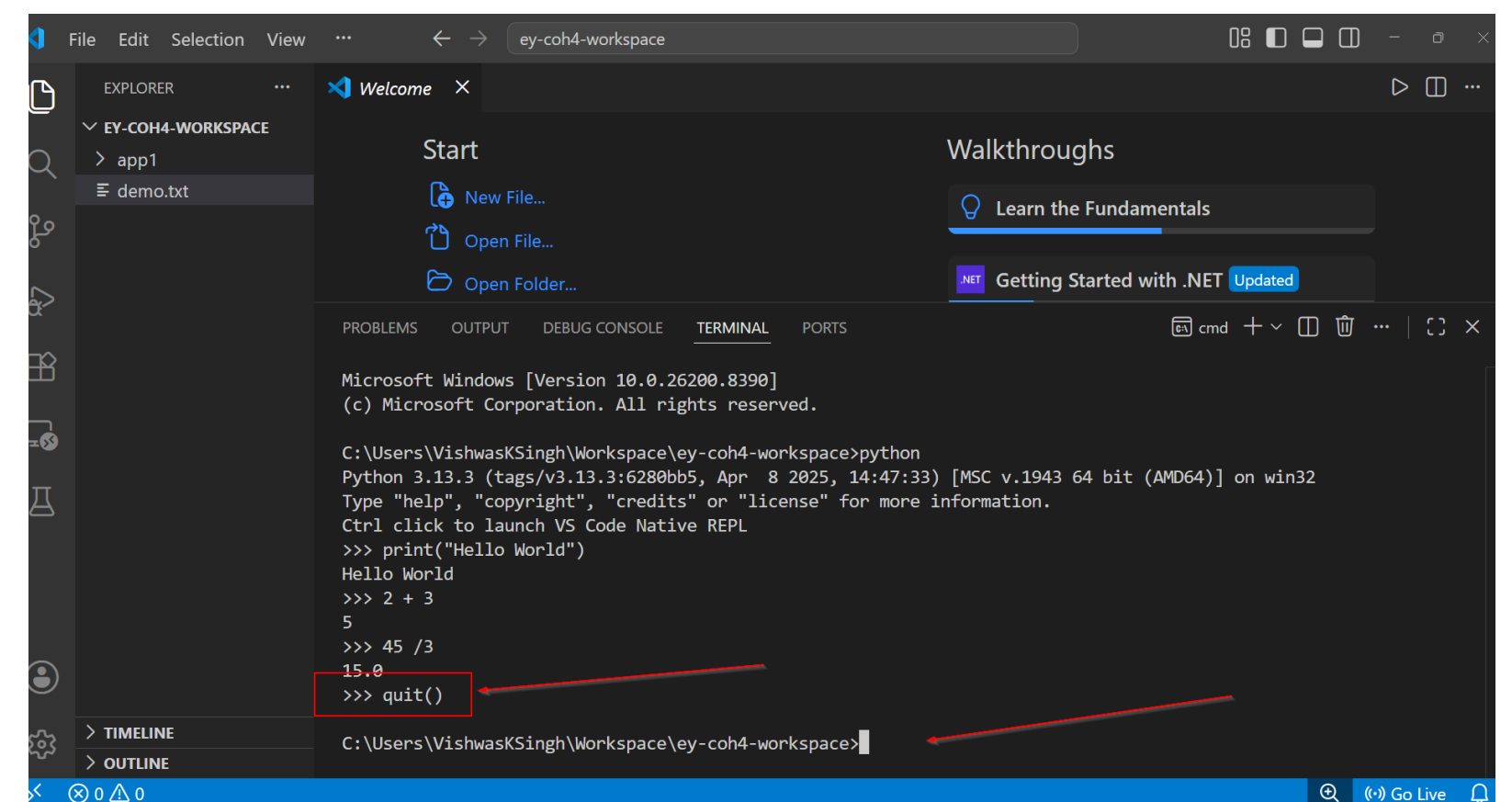
Interacting with Python

1. Interactive Mode / Prompt mode



```
Microsoft Windows [Version 10.0.26200.8390]
(c) Microsoft Corporation. All rights reserved.

C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace>python
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
Ctrl click to launch VS Code Native REPL
>>> print("Hello World")
Hello World
>>> 2 + 3
5
>>> 45 / 3
15.0
>>>
```



```
Microsoft Windows [Version 10.0.26200.8390]
(c) Microsoft Corporation. All rights reserved.

C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace>python
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
Ctrl click to launch VS Code Native REPL
>>> print("Hello World")
Hello World
>>> 2 + 3
5
>>> 45 / 3
15.0
>>> quit()
C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace>
```

We use this when we are initially creating, or we need to understand working of a small code snippet

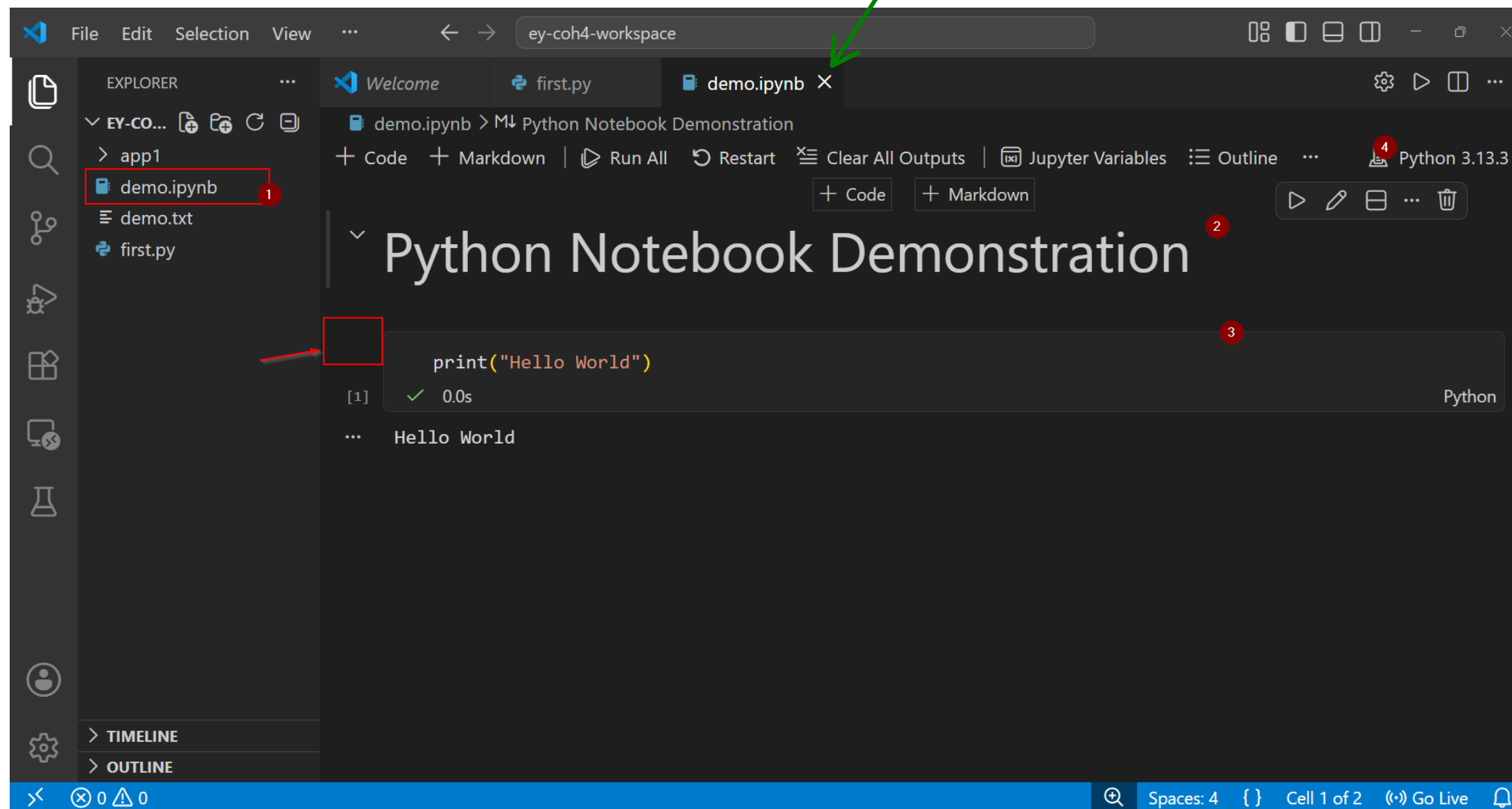


The image shows the Visual Studio Code (VS Code) interface. At the top, the Explorer sidebar on the left displays the workspace structure: 'EY-COH4-WORKSPACE' containing a folder 'app1' and a file 'demo.txt'. The file 'first.py' is selected, indicated by a red circle '1'. The main editor area shows the contents of 'first.py', which consists of two lines of Python code: '1 print("hello World")' and '2 print(4+45)'. A red circle '2' is placed next to the second line. Below the editor, the TERMINAL panel is active, showing the command 'python first.py' being executed in the shell. The output of the command is 'hello World' followed by '49' on the next line. A red circle '3' is placed next to the command. The status bar at the bottom of the window shows the current file's location as 'Ln 2, Col 12', the encoding as 'UTF-8', and the language as 'Python 3.13.3 64-bit'.

3. Notebook Mode (.ipynb)

Notes + Interactive + saved
Data Analyst/ Scientist

Jupyter Notebooks
Google Colab



Python Language Basics

Learning how to program

When ever you are given a problem
Break the problem to sub problem
Find Solution to each sub problem
Integrate it



Analyse the sub problem

Input?

Output?

What is relationship b/n Input & Output?
(Also called Logic)

Example:

- get data from various departments
- you have to compile those
- generate insights
- create a report out of it



What format is your input → Eg. Excel File

- Read the Excel File(Input)
- Clean It (Logic process)
- Format for next processing (output)

To Learn a Programming Language

1. How to store data temporarily
2. How to do Conditions
3. How to repeat (Loops and Functions)
4. What the base paradigm

Python is a object based Functional Programming Language

What are alternatives to Functional Programming Paradigm?

O – Object oriented Programming

P – Procedural Programming Approach

1. How to store data temporarily

Variables

Label to memory location

salary = 20000000
↑ ↑ ↑
variable Assignment data
identifier operator

- It can contain alphanumeric characters, underscore(_)
- It must always start with a letter or _
- no other special characters
- must not be a keyword
- meaningful (good practice)
- It is case-sensitive

salary → 0x1Ah
2000000

```
C:\Users\VishwasKSingh>python
Python 3.13.3 (tags/v3.13.3:6280b)
Type "help", "copyright", "credits" or "quit()"
>>> salary = 20000
>>> salary
20000
>>> print(salary)
20000
>>> type(salary)
<class 'int'>
>>> name = "vishwas"
>>> type(name)
<class 'str'>
>>> bonus = 2135.6789
>>> type(bonus)
<class 'float'>
>>> |
```

```
False
None
True
and
as
assert
async
await
break
class
continue
def
del
elif
else
except
finally
for
from
global
if
import
in
is
lambda
nonlocal
not
or
pass
raise
return
try
while
with
yield
```

Keywords

Data Type

- Int (1234)
- Float (12.34)
- String ("12.34")
- Bool (True)
- Complex (2+3j)

```
>>> k = 1234
>>> k
1234
>>> type(k)
<class 'int'>
>>> k1 = 12_00_234
>>> type(k1)
<class 'int'>
>>> k2 = 0b1010
>>> k2
10
>>> type(k2)
<class 'int'>
>>> k3 = 0x0A
>>> k3
10
>>> type(k3)
<class 'int'>
>>> k4 = 0o27
>>> k4
23
>>> type(k4)
<class 'int'>
>>> k5 = complex(2,3)
>>> k5
(2+3j)
>>> type(k5)
<class 'complex'>
>>>
```

```
>>> f1 = 12.3456
>>> type(f1)
<class 'float'>
>>> f2 = 123456e-4
>>> f2
12.3456
>>> type(f2)
<class 'float'>
>>>
```

```
>>> f3 = True
>>> f3
True
>>> type(f3)
<class 'bool'>
>>> f4 = False
>>> type(f4)
<class 'bool'>
>>> f4
False
>>>
```



```
>>> s1 = "vishwas"
>>> s1
'vishwas'
>>> type(s1)
<class 'str'>
>>> s2 = 'vishwas'
>>> s2
'vishwas'
>>> type(s2)
<class 'str'>
>>> s3 = '''vishwas'''
>>> s3
'vishwas'
>>> type(s3)
<class 'str'>
>>> s4 = """vishwas"""
>>> s4
'vishwas'
>>> type(s4)
<class 'str'>
>>> s5 = "This is Vishwas's book"
>>> s6 = '''
... This is an example of
... multiline value. It is considered
... as a single string itself
... '''
>>> s6
'\nThis is an example of\nmultiline value. It is considered\nas a single string itself\n'
>>>
```

```
>>> s7 = "\nI am Vishwas K Singh\n I am Learning python"
>>> s7
'\nI am Vishwas K Singh\n I am Learning python'
>>> print(s7)

I am Vishwas K Singh
 I am Learning python
>>> s8 = "C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace\first.py"
File "<python-input-65>", line 1
    s8 = "C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace\first.py"
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
SyntaxError: (unicode error) 'unicodeescape' codec can't decode bytes in position 2-3: truncated \UXXXXXXX escape
>>> s8 = "C:\\Users\\VishwasKSingh\\Workspace\\ey-coh4-workspace\\first.py"
>>> s8
'C:\\Users\\VishwasKSingh\\Workspace\\ey-coh4-workspace\\first.py'
>>> print(s8)
C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace\first.py
>>> s8
'C:\\Users\\VishwasKSingh\\Workspace\\ey-coh4-workspace\\first.py'
>>> s9 = r"C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace\first.py"
>>> s9
'C:\\Users\\VishwasKSingh\\Workspace\\ey-coh4-workspace\\first.py'
>>> name = "vishwas"
>>> greet = f"Hey there!, Welcome to session {name}"
>>> greet
'Hey there!, Welcome to session vishwas'
>>> version = 3
>>> greet = f"Hey there!, Welcome to session {name}, I am learning python {version}"
>>> greet
'Hey there!, Welcome to session vishwas, I am learning python 3'
>>> salary = 123.4567890
>>> greet = f"Hey there!, Welcome to session {name}, I am learning python {version}, I am earning a salary of {salary:.2f}"
>>> greet
'Hey there!, Welcome to session vishwas, I am learning python 3, I am earning a salary of 123.46'
>>> |
```

Operators

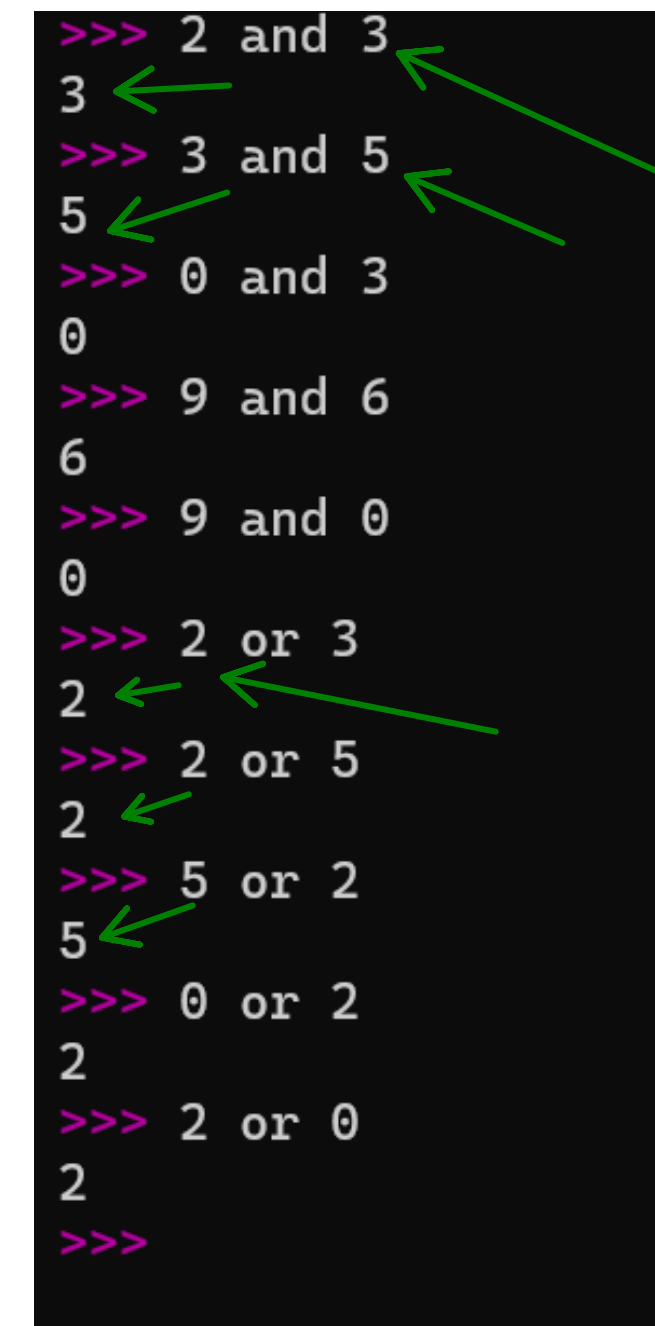
- Assignment (`=, +=, -=, *=...`)
- Arithmetic (`+, -, *, /, %, //(integer division), ** (power)`)
- Logical (`and, or, not`)
- Relational (`<, >, ≤, ≥, ≠`)
- Identity (`as`)
- Membership (`in, not in`)
- Bitwise (`&, |, ~`)

Pointers:

- A 0(zero) is always False
- Any non-zero number is always True

For faster evaluation python uses "Short Circuit Evaluation"

- In a logical expression `expr1 and expr2`
`expr2` is not evaluated unless `expr1(True)`
- In a logical expression `expr1 or expr2`
`expr2` is not evaluated at all when `expr1` is already True



```
>>> 2 and 3
3
>>> 3 and 5
5
>>> 0 and 3
0
>>> 9 and 6
6
>>> 9 and 0
0
>>> 2 or 3
2
>>> 2 or 5
2
>>> 5 or 2
5
>>> 0 or 2
2
>>> 2 or 0
2
>>>
```

Arrows in the original image indicate short-circuit evaluation: for `2 and 3`, the result `3` is the value of the second operand; for `0 and 3`, the result `0` is the value of the first operand; for `2 or 3`, the result `2` is the value of the first operand; for `5 or 2`, the result `5` is the value of the first operand.

"Guard Pattern"

```
>>> x = 10
>>> y = 2
>>> x / y
5.0
>>> y = 0
>>> x / y
Traceback (most recent call last):
  File "<python-input-94>", line 1, in <module>
    x / y
    ~^~
ZeroDivisionError: division by zero
>>> y != 0 and x / y
False
>>> y = 2
>>> y != 0 and x / y
5.0
>>> |
```

Lab: GST Calculation v1.0

The GST on Electronic Appliances was reduced from 28% to 18%. If an AC costed Rs.30000(excluding GST) what are the values of GST at both the rates & what is the difference in cost (how much you would save now)?

Assume no other taxes are applied

Console I/O in python

- printing to a console - `print(...)`
- taking values from a console - `input(...)`



Input Prompt
i.e message for the user

Pointers:

1. Whatever you take from an input function is always a string
2. You as a developer have to convert to right data type

```
>>> age = input("Enter your age: ")
Enter your age: 56
>>> age
'56'
>>> type(age)
<class 'str'>
>>> age = int(age)
>>> age
56
>>> type(age)
<class 'int'>
>>>
```

Lab: GST Calculation v2.0

gst_calcv2.py > ...

```
1  # Input Section
2  ac_cost = float(input("Enter the AC Price: "))
3  old_gst = float(input("Enter old GST Rate: ")) / 100 # Calculating actual value for 28%
4  new_gst = float(input("Enter new GST Rate: ")) / 100
5
6  # Logic or calculation
7  old_total_cost = ac_cost + (ac_cost * old_gst)
8  new_total_cost = ac_cost + (ac_cost * new_gst)
9
10 total_savings = old_total_cost - new_total_cost
11
12 # Output
13 print(f"The Cost Saving of a AC at Rs.{ac_cost} with reduction in GST from {old_gst * 100:.2f}% to {new_gst * 100}%\
14 | is Rs.{total_savings} ")
15
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
C:\Users\VishwasKSingh\Workspace\ey-coh4-workspace>python gst_calcv2.py
Enter the AC Price: 25000
Enter old GST Rate: 28
Enter new GST Rate: 20
The Cost Saving of a AC at Rs.25000.0 with reduction in GST from 28.00% to 20.0% is Rs.2000.0
```

Summary Day 1

We learnt about

1. What is programming How to approach it
 2. Different kinds of Programming Language
 3. Features of Python Programming Language, different versions, different interpreters
 4. Variables, Data Types & Operators, Console I/O in python
-

What to look out for tomorrow?

- Conditionals
- Loops
- Strings, Lists, Dicts, Tuples, Sets
